NEXT.md の同時更新

判断の食い違いと古い状態の上書き
最小運用の設計と小規模実験

杉田研治

Version 0.1.0

2026-11-07

2. 共有する作業の記録

現在の状態・判断・残課題を3ファイルに分ける

NEXT.mdの技術セミナー資料では、会話をまたいで作業を続けるための記録を紹介しています。

ファイル	残す内容	今回の例
`NEXT.md`	現在の状態、次の作業、再開時の注意	実装完了、検証待ち
`DECISIONS.md`	判断と理由、継続する仕様・制約	採用する保存方式、対応するOS
`backlog.md`	後で取り組む課題、問題点、保留事項	将来の追加検証、運用の改善

共通の扱い方は `AGENTS.next.md` に定めています。

今回は、複数の作業者が進めた内容を、この3ファイルへどう集約するかを考えます。

3. 同時に作業すると起きること

文字の競合と、内容の食い違いを確認する

二人が同じ状態を読んで作業を始めても、途中で成果や判断が分かれます。

問題	具体例	必要な確認
古い状態の上書き	Bの報告を採用したら、Aの実装完了が消えた	先行する成果を保持しているか
判断の対立	AはSQLite、BはJSONを採用したい	何を優先し、誰が決めるか
前提の変化	旧仕様の検証合格を、新仕様にも適用した	何の版・対象を検証したか

Gitで競合なくマージできても、記録の意味や前提が一致するとは限りません。

以下の例は、問題を再現するために用意した架空の作業です。

4. 例1：古い全文で上書きする

後から届いた報告が、先に終わった成果を消す

AとBは、どちらも「実装・説明書ともに未完了」の状態から始めました。

順序	操作	NEXT.mdの実装	NEXT.mdの説明書
開始時	AとBが同じ状態を読む	未完了	未完了
Aの作業後	実装完了を記録する	**完了**	未完了
Bの作業後	古い状態に説明書完了を追記し、全文を採用する	**未完了に戻る**	完了

Bの報告は、Aが実装を終えたことを反映していません。

更新担当が1名でも、古い全文を順番に貼り付けると同じことが起きます。

5. 例1への対処

最新状態へ、対象の成果だけを取り込む

Bは「説明書の確認が完了した」という報告を、独立したファイルに残します。

照合するもの	内容
最新の共有記録	Aによる実装が完了している
Bの報告	説明書の確認が完了した。作業開始時の基準は古い
判断	説明書の完了は、実装完了を取り消す理由にならない
取り込み後	**実装完了と説明書完了の両方を保持する**

基準が古いことは、最新状態を読み直して照合する合図になります。

独立した成果なら、古い基準の報告でも取り込めます。

6. 例2：判断が食い違う

同じ状態を読んでも、重視する条件が異なる

共通の基準では、データの保存方式はまだ決まっていません。

作業者	提案	重視した理由
A	SQLiteを採用する	検索しやすくしたい
B	JSONを採用する	配布を簡単にしたい

両者が `DECISIONS.md` の「保存方式」を直接変更すると、Gitで文字の競合が起きました。

どちらの行を残すか選ぶだけでは、要件の優先順位を決めたことになりません。

共通の基準コミットは、判断の一致を保証しません。

7. 例2への対処

両案と理由を残し、依存する作業を保留する

独立した報告ファイルなら、両方の提案をGit上で共存させられます。

記録する場所	残す内容
A・Bの個別報告	提案、理由、基準、根拠
`DECISIONS.md`	保存方式は未決。両案と理由、決める人、確認すべき優先要件
`NEXT.md`	優先要件を確認する。保存方式に依存する実装は保留

共有記録を編集する**統合担当**と、要件の優先順位を決める**判断担当**を区別します。

対立する判断を残しておけば、独立した作業を続けながら必要な判断を待てます。

8. 例3：競合なしで前提が食い違う

新しい仕様と、古い仕様の検証結果が混ざる

開始時は、仕様v1の実装・説明書・Linuxでの検証が完了しています。

作業者	更新したファイル	変更内容
A	`NEXT.md`	v1の検証合格を基に「リリース可能」と記録
B	`DECISIONS.md`	仕様をv2へ変更し、Windows対応を必須にする

別のファイルへの変更なので、Gitは**競合なくマージできました。**

しかし、共有記録には「現行仕様はv2」と「v1の検証でリリース可能」が並びます。

Windowsを追加したv2の検証は、まだ終わっていません。

9. 例3への対処

過去の成果を保持し、現行仕様の未完了を明示する

「完了」が、どの仕様・版・対象に対するものかを照合します。

項目	取り込み後の記録
v1の実装・説明書	完了を保持する
v1 / Linuxの検証	合格した過去の成果として残す
現行仕様	v2 / Linux + Windows
v2の検証	**未完了。追加要件を検証する**
リリース判定	**保留**

v1の成果を消す必要はありません。v2にも合格したという読み違いを防ぎます。

状態の更新では、成果と、その成果が成立した前提を一緒に残します。

10. 最小限の4ルール

報告・更新担当・照合・未決の扱いを決める

ルール	実施すること
1. 個別の報告を残す	基準、対象と前提、成果と根拠、変更提案を書く
2. 更新担当を1名にする	共有3ファイルは統合担当が更新する
3. 最新状態と照合する	古い全文を採用せず、対象事項だけを取り込む
4. 対立を未決として残す	両案と理由を保存し、依存作業を保留する

更新担当が1名でも、照合を省けば古い状態を上書きします。

4つを組み合わせて、成果と判断理由を守る運用にします。

11. 作業者が残す報告

何を根拠に、何を変更したいかを短く書く

例： `reports/B-20261107-01.md` に、説明書の作業結果を記録します。

項目	記入例
作業者	B
基準コミット	作業を始めた時点のコミットID
対象・前提	v1の説明書
観測・根拠	記載内容と動作を照合済み。確認記録の保存先を添える
変更提案	NEXT.mdの説明書の状態を完了へ
判断への影響	実装の状態や保存方式は変更しない

「作業を完了した事実」と「共有記録への変更提案」を書き分けます。

12. 統合担当の取り込み手順

最新状態から変更案を作り、確定前に再確認する

1. **共有先の最新状態を読む。** 未コミットの編集があれば取り込みを止め、状態を確認する。
2. **報告を照合する。** 基準以降の変更、成果の対象、他の判断への影響を見る。
3. **変更案を別の下書きに用意する。** 成果を保持し、対立は未決に残す。
4. **反映直前に基準を再確認する。** 変わっていれば読み直して照合する。
5. **3ファイルを一組で確認し、確定する。** 一つのコミットにまとめて通常pushする。

公開を拒否された場合は、最新状態を確認してやり直します。

統合担当が書きかけの状態と、共有先で確定した状態を区別します。

13. 小規模実験の方法

固定した7事例で、問題と運用案を再現する

実験の要素	実施したこと
環境	一時領域に独立したGitリポジトリを作成
操作	書き込み、コミット、ブランチ、merge、ローカルクローン、push
比較・確認	直接更新との比較、古い基準・編集中の記録・公開競合の確認
検査	成果の保持、未決の維持、拒否時の状態不変、Gitの終了コード
証拠	共有記録、個別報告、差分、履歴、結果JSONを保存

意味の照合結果は、各事例の前提から定めてスクリプトに与えました。

実装・OS検証などの成果は架空の設定です。複数人・複数AIによる実作業は試していません。

14. 実験結果：成果と判断の保持

事例01〜04で確認した挙動

事例	問題・条件	運用案で確認した結果
01 古い全文	Aの実装完了が消える	A・B両方の成果を保持
02 判断の対立	SQLiteとJSONで文字の競合	両案を保存し、未決と依存作業の保留を維持
03 前提の変化	v2仕様とv1の合格が無競合で混在	v1の成果を残し、v2を再検証待ちにする
04 遅い報告	v2への変更後にv1の完了が届く	古い基準での適用を止め、対象を明示して再照合

古い報告を捨てるのではなく、現行状態で何を意味するかを確認できました。

この結果は、事例ごとに定めた照合判断を使った確認です。

15. 実験結果：変更の検知と公開

事例05〜07で確認した挙動

事例	条件	確認した結果
05 未コミット編集	HEADは同じだが共有記録を編集中	適用を止め、既存の編集を保持
06 照合後の更新	適用前に別のコミットが入る	基準の変化を検知し、再照合して両成果を保持
07 公開の競合	別クローンから古い履歴をpush	通常pushが拒否され、共有先の先行更新を保持

全7事例で、定義した期待結果を確認しました。

7件は確認した事例の数です。未知の案件への成功率や、自動で矛盾を発見する能力を示す数値ではありません。

16. 適用条件と残る限界

今回の案で扱える範囲を明確にする

適用条件	残る限界
作業者と統合担当の作業場所を分ける	同じ場所への自由な同時書き込みは保護しない
共有記録の更新担当を1名にする	運用上の約束であり、ロックで強制していない
コミット済みの基準から照合する	確認と書き込みの間の割り込みは排除できない
共有先の確定コミットを読む	書き込み途中の中断・担当交代の復旧は未検証
根拠と前提を読んで判断する	人やAIによる意味の見落としは残る

今回の結果は、最小運用を次の試行へ進めるための材料です。

17. 次は実際の作業者で試す

固定した回答を渡さず、運用の負担と見落としを確認する

独立した2作業者と統合担当で、短い作業を進める試行を検討します。

観測すること	確認したい点
報告の記入負担	基準・対象・根拠を無理なく残せるか
統合時間	照合や保留が作業の停滞につながらないか
前提の見落とし	何の仕様・版に対する成果か読み取れるか
不要な保留	独立した成果を取り込めるか
既存情報の保持	無関係な状態や判断理由を消さないか

有効性と負担を確認してから、適用範囲と正式導入を判断します。

18. まとめ・質疑

成果と判断理由を残し、次の作業へつなぐ

- **更新者を1名にすることと、最新状態との照合を組み合わせる。** 古い全文を採用せず、独立した成果を対象事項ごとに取り込みます。
- **Gitの競合に加え、判断や前提の食い違いを見る。** 「何の版・対象に対する完了か」を確認します。
- **判断の対立は両案を残し、決める人と次の確認を明確にする。** 依存する作業を保留し、独立した作業を続けられるようにします。

7事例で手順を確認しました。次は、実際の作業者による有効性と負担を確かめます。

参照資料：`DESIGN.md`、`REPORT_TEMPLATE.md`、`results/run-02/RESULTS.md`